GitHub 週間スター上位レポート

2026-11-02 作成 ／ 対象: 2026-10-26 以降に作成されたリポジトリ
テーマ: WebLLM

#	区分	リポジトリ	★	一言で
1	テーマ	soumwadeep/ghostlight	1	死者の日に故人を偲ぶ文章と祭壇を、ブラウザ内のGemma 2で生成するローカルファーストなWebアプリ
2	テーマ	lf735/webllm	0	Google Gemma 4 系モデルをブラウザ（WebGPU）または Ollama 経由でローカル実行するチャット用 Web UI
3	テーマ	geohacker/monty-webllm	0	WebLLM を使ってブラウザ上の LLM で Montandon データに問い合わせることを目指した、Vite 製の小規模な実験リポジトリ
4	テーマ	nick-montana/WebLLM-Gateway	0	ブラウザ内で動くWebLLMを使ってAIエージェント作りを学ぶための、初期段階の教材リポジトリ
5	テーマ	Hazz-i/WebLlmBridge	0	ブラウザでログイン中の Claude.ai や ChatGPT などの Web チャットを、OpenAI 互換のローカル API として使えるようにするブリッジです。
6	総合	deepseek-ai/dsh-superagent-prototype	8,297	DeepSeek Harness 向けバンドルとして動く、単一 GitHub リポジトリ専属の能動型コーディングエージェントの研究プロトタイプ
7	総合	anhao/ai-pose-gallery-zh	5,174	日本の作者ギア氏のAI絵画用ポーズ・服装・人物参考画像集を中国語化・統合し、生成AI連携ワークベンチとComfyUIプラグインを加えたプロジェクト
8	総合	heydamir/unslop	4,954	AIが作ったっぽいUIを、デザイナーのレビューのように段階的に直して個性あるデザインに仕上げる Claude Code / Codex 向けスキル
9	総合	wangzan101/Mika	4,007	話しかけるだけで、Androidのアプリやウェブを裏画面で操作して用事を片付けてくれるオンデバイスAIエージェントです。
10	総合	valentderah/totk-pc-recomp	3,245	Switch版『ゼルダの伝説 ティアーズ オブ ザ キングダム』を、エミュレーションではなく静的リコンパイルでWindows/Linux向けネイティブx86-64アプリにするプロジェクト

テーマ 1. soumwadeep/ghostlight

https://github.com/soumwadeep/ghostlight

★ 1 ／ フォーク 0 ／ 言語 TypeScript ／ 作成 2026-10-27 ／ MIT
該当テーマ: WebLLM

死者の日に故人を偲ぶ文章と祭壇を、ブラウザ内のGemma 2で生成するローカルファーストなWebアプリ

概要

Ghostlightは、大切な故人について6つの質問に答えると、追悼文（remembrance）、インタラクティブな祭壇（ofrenda）、オフラインで開ける記念HTML（keepsake）を作るアプリです。文章生成はGemma 2（2B）をWebLLMとWebGPUでブラウザ内実行し、データはIndexedDBに保存されるため、バックエンドやクラウド推論、アカウントは一切ありません。質問への回答はまずアーカイブから検索し、該当がなければ回答を拒否して引用を必ず示す設計で、小型モデルのハルシネーション対策を明示しています。Hacktoberfest 2026 DEV Challenge「Ghost in the Machine」向けに作られた作品です。

要点

- Gemma 2 2BをWebLLM＋WebGPUでブラウザ内実行
- バックエンド・APIキー・環境変数が一切不要
- 検索優先で根拠がなければ回答を拒否する設計
- モデル不要のキャンドルモードやOllama連携も選択可
- PWA対応で初回DL後は完全オフライン動作

何をするものか

Día de Muertos（死者の日）をモチーフにした追悼のためのスタジオです。料理、歌、場所、口癖、小さな物、やり残したことという6つの質問（six rites）に答えると、回答をもとに三人称で書かれた150〜200語・3段落の追悼文が生成されます。さらに6本のロウソク、マリーゴールド、水、パペル・ピカド、回答から抽出した供物を並べた段組みの祭壇が表示され、ロウソクや供物をクリックすると元の記憶を読めます。最後に、スクリプトを含まない単一HTMLファイルとして記念品を書き出せます。READMEでは、霊媒やセラピーの代わりではなく「言葉を置く場所」であると明記されています。

主な機能

ブラウザ内のGemma 2による追悼文生成、アーカイブに基づく質問応答（Ask the ofrenda）、複数の祭壇を管理できるローカルアーカイブ（JSONインポート・エクスポート、ワンクリック削除）、オフラインで開ける記念HTMLの出力があります。モデルをダウンロードしたくない場合は、ユーザー自身の文章を書き換えずに並べるだけの「Candle mode」が使えます。ローカルでOllamaを動かしているなら任意のモデルを使うオプションもあり、接続できなければクラウドではなく記憶検索にフォールバックします。音声入力はベンダーへ音声が送られる可能性があるため意図的に搭載していません。キーボード操作、フォーカス表示、reduced-motion対応などアクセシビリティにも配慮されています。

仕組み・アーキテクチャ

処理の流れは、6つの回答をIndexedDBのアーカイブに保存し、記憶検索で事実を確定させてから、Gemma 2で追悼文を生成し、祭壇と記念品を作るというものです。AIの役割は2つに分かれています。追悼文の生成では、記憶のみから書き、各段落を特定の記憶に割り当てるルールを課しています。質問応答ではまず記憶を検索し、一致がなければ「The flame has no memory of that.」と拒否し、一致した場合もモデルは言い換えのみを行い、引用はモデルではなくアプリ側が付与します。プロンプトは src/lib/ai/prompts.ts にあり、Aboutページにそのまま表示されます。src/lib/ai 以下に retrieval、candle、weave、ask、webllm、ollama、parse などのモジュールが分かれています。

技術構成

React 19、Vite 8、TypeScript 6をベースに、ルーティングに react-router-dom、アニメーションに motion、アイコンに lucide-react、IndexedDB操作に idb-keyval、モデル実行に @mlc-ai/web-llm、PWA化に vite-plugin-pwa を使っています。フォントは Fraunces、Instrument Sans、IBM Plex Mono（fontsource経由）。テストは vitest、リントは oxlint、検証スクリプトに playwright が含まれています。モデルIDは gemma-2-2b-it-q4f16_1-MLC で、約1.4GB、約2GBの空きGPUメモリが必要です。Gemma 3はWebLLM 0.2.86のビルド済みレコードが読み込めない上流の問題（issue #873）があるため採用を見送ったと説明されています。Node.jsは22以上が必要です。

使い方・導入方法

git clone 後に npm install、npm run dev を実行し、http://localhost:5173 を開くだけで動きます。環境変数やAPIキー、外部サービスの設定は不要です。生成画面では、約1.4GBのGemma 2を呼び出す方法（デスクトップのChromeかEdgeが推奨）か、ダウンロード不要のCandle modeかを選べます。npm test で20個のユニットテスト、npm run typecheck で型検査、npm run build で本番ビルドとService Worker生成、npm run verify:gemma でモデルの実動作確認ができます。ビルド成果物は静的サイト（dist/）なので、render.yaml を使ったRenderへのワンクリックデプロイや、未知パスを index.html にリライトできる任意の静的ホストで公開できます。

注目ポイント・注意点

「ローカルファーストには境界がある」として、端末外に出るもの（静的ページのリクエストと一度きりのモデルダウンロード）と出ないもの（回答、写真、質問、生成物）を表で明示している点が誠実です。一方で、共有PCやブラウザ拡張、ロック解除された端末からは守れないこと、サイトデータを消すとアーカイブも消えることも書かれています。2Bモデルは小さく、アーカイブにない細部を書いてしまう場合があるため、記憶を常に表示し、厳密なCandle modeを用意しています。デモの人物 Ruma Das は架空で、肖像も生成画像、デモの追悼文はモデル出力ではなく手書きの目標文です。ブラウザLLMで根拠付き応答を実装する参考例として読む価値があります。

技術構成: TypeScript、React 19、Vite 8、WebLLM (@mlc-ai/web-llm)、WebGPU、Gemma 2 2B IT、IndexedDB (idb-keyval)、vite-plugin-pwa、react-router-dom、motion、lucide-react、Vitest、oxlint、Playwright、Ollama（任意）、Render
向いている人: ブラウザ内LLM（WebLLM/WebGPU）でプライバシーを守るアプリを作りたいフロントエンドエンジニア、検索優先・引用付きでハルシネーションを抑える設計を学びたい人、ローカルファーストなPWAやアクセシブ

ルなUIの実装例を探している人に向いています。また、データを外部に出さずに故人の思い出を文章として残したい一般ユーザーの利用にも適しています。

テーマ 2. lf735/webllm

https://github.com/lf735/webllm

★ 0 ／ フォーク 0 ／ 言語 JavaScript ／ 作成 2026-10-26
該当テーマ: WebLLM

Google Gemma 4 系モデルをブラウザ（WebGPU）または Ollama 経由でローカル実行するチャット用 Web UI

概要

Gemma 4 Studio は、Google Gemma 4 ファミリー（E2B、E4B、12B、26B MoE など）を、クラウドにデータを送らず手元のマシンで動かすための Web インターフェースです。実行エンジンは 2 種類から選べます。1 つは @mlc-ai/web-llm による WebGPU でのブラウザ内完結の実行、もう 1 つは Ollama のローカルサーバー（localhost:11434）への接続です。Thinking Mode の思考過程の可視化、画像入力、tok/s や TTFT のリアルタイム表示など、ローカル LLM を試す人向けの機能がまとまっています。

要点

- WebGPU のブラウザ内実行と Ollama 連携の2エンジンを選べる
- 軽量モデルはブラウザ内だけで動き、データは外部に出ない
- <thought> タグを検出して思考過程を折りたたみ表示する
- tok/s・TTFT・トークン数をリアルタイムに表示する
- Vite と素の JavaScript による単一ファイル構成の小規模実装

何をするものか

Gemma 4 Studio は、Google の Gemma 4 系モデルをローカル環境で対話的に使うためのチャット型 Web UI です。README は README 自体がフランス語で書かれており、「ローカルかつプライベートな AI インターフェース」を掲げています。会話データをクラウドに送らずに済むのが大きな特徴です。軽量な Edge 向けモデルはブラウザ内だけで推論します。より大きなモデルは、手元で動かしている Ollama サーバーに処理を任せる二段構えになっています。リポジトリのメタデータには説明文、トピック、ライセンス情報がなく、スターやフォークも 0 の個人プロジェクトです。

主な機能

README に記載されている主な機能は次のとおりです。 ・Thinking Mode の可視化：Gemma 4 の <thought> タグをリアルタイムに検出し、タイマーと思考トークン数つきのアニメーション付きアコーディオンで表示します。 ・マルチモーダル入力：画像をドラッグ＆ドロップまたは添付して、モデルに解析させられます。 ・テレメトリ HUD：tok/s、TTFT（ms）、総トークン数を表示します。 ・会話管理：履歴を localStorage に自動保存し、複数セッションの作成やワンクリック削除ができます。 ・その他：会話を .md 形式でエクスポートできます。コードブロックはハイライトとコピーボタンつきで表示されます。 ・パラメータ調整：Temperature、Top-P、最大トークン数を変更できます。システムプロンプトのプリセット（Codeur、Raisonneur、Synthétique、Polyvalent）も用意されています。

仕組み・アーキテクチャ

実行エンジンは 2 種類あります。1 つ目は WebGPU モードです。@mlc-ai/web-llm を使い、Gemma 4 E2B や Gemma 2 2B といった超軽量モデルを 100% ブラウザ内で実行します。2 つ目は Ollama モードです。localhost:11434 の REST API にストリーミングで接続し、E4B、12B、26B MoE、31B といった大きめのモデルを扱います。この場合は CUDA、Metal、Vulkan による高速化を Ollama 側で利用します。README では LM Studio も選択肢として挙げられていますが、具体的な接続方法は README からは不明です。ファイル構成は index.html、src/main.js、style.css の 3 つが中心です。フレームワークを使わない素の JavaScript で、ロジックの大半は main.js にまとまっていると見られます。

技術構成

言語は JavaScript（ES Modules）で、ビルドツールと開発サーバーには Vite（^8.3.5）を使っています。ランタイムの依存パッケージは @mlc-ai/web-llm（^0.2.86）の 1 つだけです。UI フレームワークは package.json から確認できず、HTML、CSS、JavaScript による実装と考えられます。コードハイライトや Markdown のレンダリングに外部ライブラリを使っているかどうかは、マニフェストからは確認できません。npm scripts は dev、build、preview の 3 つです。dev は vite --host 0.0.0.0 --port 5173 で起動するため、LAN 内の他の端末からもアクセスできる設定になっています。

使い方・導入方法

リポジトリのフォルダで npm install と npm run dev を実行すると、http://localhost:5173 で UI が開きます。WebGPU モードを使うには、Chrome、Edge、Firefox Nightly など WebGPU に対応したブラウザが必要です。Ollama モードは次の手順で使います。 1. OLLAMA_ORIGINS="*" ollama serve で、CORS を許可した状態でサーバーを起動します。 2. ollama run gemma4:latest などでモデルを取得します。VRAM に応じて gemma4:e2b、e4b、12b、gemma2:2b も選べます。 3. UI の Ollama タブを選ぶと、インストール済みモデルが自動検出されます。 設定（⚙️）メニューからは接続テストも行えます。

注目ポイント・注意点

ブラウザ内推論とローカルサーバー推論を 1 つの UI で切り替えられるため、ローカル LLM の比較や学習に使いやすい構成です。一方で、いくつか注意点があります。 ・ライセンスが不明です。package.json には ISC とありますが、リポジトリとしてのライセンスは明記されていません。 ・OLLAMA_ORIGINS="*" はすべてのオリジンを許可する設定です。共有ネットワークで使う場合はセキュリティに配慮が必要です。 ・dev サーバーも 0.0.0.0 で公開される設定です。 ・Gemma 4 の各モデル名や、web-llm でのモデル対応状況が実際に利用できるかは、README からは検証できません。 ・テストコードやドキュメントはほとんどなく、実験的な個人プロジェクトと捉えるのが妥当です。

技術構成: JavaScript、Vite、@mlc-ai/web-llm、WebGPU、Ollama REST API、HTML/CSS、localStorage
向いている人: クラウドにデータを送らずに Gemma 系モデルを試したい開発者や学生に向いています。WebGPU によるブラウザ内 LLM 推論の実装例を学びたい人や、Ollama 用に軽量なフロントエンドを探している人にも参考になります。Thinking Mode の可視化や tok/s 計測でモデルの挙動や性能を比較したい用途にも適しています。

テーマ 3. geohacker/monty-webllm

https://github.com/geohacker/monty-webllm

★ 0 ／ フォーク 0 ／ 言語 JavaScript ／ 作成 2026-10-27
該当テーマ: WebLLM

WebLLM を使ってブラウザ上の LLM で Montandon データに問い合わせることを目指した、Vite 製の小規模な実験リポジトリ

概要

このリポジトリは、説明文によれば WebLLM をベースに Montandon データへの問い合わせ（data interrogation）を行うためのものです。依存パッケージは @mlc-ai/web-llm と Vite のみで、ブラウザ内で言語モデルを動かすフロントエンドの実験と考えられます。ただし README は見出しだけで、具体的な機能、対象データの形式、使い方は書かれていません。package.json の名前が webllm-experiments であることからも、試作段階のプロジェクトと受け取るのが妥当です。

要点

- 説明文は「WebLLM based data interrogation for Montandon data」
- 依存は @mlc-ai/web-llm と Vite のみで、構成は最小限
- README はタイトルのみで、詳細は README からは不明
- ビルド済みの dist ディレクトリと .env.example がコミットされている
- スター・フォークは 0、ライセンスは不明の個人実験段階

何をするものか

リポジトリの説明文は「WebLLM based data interrogation for Montandon data」です。WebLLM を使って Montandon データに対して質問や分析を行うツールを目指していると読み取れます。WebLLM は、ブラウザ上で大規模言語モデルを動かすためのライブラリです。一方で、Montandon データの中身や形式、データの取得元、どのような問い合わせができるかは README からは不明です。README には見出し「monty-webllm」しかなく、目的は説明文とファイル構成から推し量るしかありません。

技術構成

package.json に記載された実行時の依存は @mlc-ai/web-llm（^0.2.86）だけです。開発用の依存は vite（^8.3.5）のみで、"type": "module" により ES Modules として書かれています。フレームワーク（React や Vue など）は使っておらず、index.html と main.js による素の JavaScript 構成と見られます。主要言語も JavaScript です。パッケージ名は webllm-experiments で、private 指定、バージョンは 0.0.0 となっています。

仕組み・アーキテクチャ

ファイルはごく少数で、エントリポイントの index.html、ロジック本体と思われる main.js、ビルド設定の vite.config.js で構成されています。@mlc-ai/web-llm を使う構成なので、モデルの推論はサーバーではなくブ

ラウザ側で行う設計と考えられます。.env.example があることから、何らかの環境変数（API エンドポイントや設定値など）を使う可能性があります。ただし具体的な変数名、データの読み込み方法、プロンプトの設計などは、提示された情報からは不明です。

使い方・導入方法

README に手順はありませんが、package.json のスクリプトから一般的な Vite プロジェクトの流れが読み取れます。まず npm install で依存をインストールします。次に npm run dev で開発サーバーを起動します。本番用のビルドは npm run build、ビルド結果の確認は npm run preview です。.env.example を .env にコピーして値を設定する必要があると思われますが、設定すべき内容は README からは不明です。また WebLLM はブラウザ側の対応環境（WebGPU など）を必要とするため、利用前に WebLLM 側のドキュメントを確認するのが安全です。

注目ポイント・注意点

LLM をブラウザ内で完結して動かし、特定のデータセットへの問い合わせに使うという試みは、サーバー費用やデータ送信を抑えたい場面で参考になる可能性があります。一方で注意点も多くあります。ドキュメントがほぼ空で、ライセンスが不明です。ビルド成果物の dist ディレクトリがリポジトリに含まれています。スター・フォークも 0 です。利用や再配布にあたってはライセンスの扱いに注意が必要です。コードを読む前提の参考資料として扱うのが現実的です。

技術構成: JavaScript、@mlc-ai/web-llm、Vite
向いている人: WebLLM を使ってブラウザ内で LLM を動かし、特定のデータに問い合わせる仕組みを試したい開発者向けです。Montandon データに関心がある人や、最小構成の WebLLM サンプルを読みたい人にも向いています。ドキュメントがほとんどないため、ソースコードを自分で読み解ける中級者以上に適しています。

テーマ 4. nick-montana/WebLLM-Gateway

https://github.com/nick-montana/WebLLM-Gateway

★ 0 ／ フォーク 0 ／ 言語 HTML ／ 作成 2026-10-26
該当テーマ: WebLLM

ブラウザ内で動くWebLLMを使ってAIエージェント作りを学ぶための、初期段階の教材リポジトリ

概要

WebLLMを使い、ブラウザのタブ内だけで動くAIエージェントを作るための土台となるリポジトリです。README では、サインアップやAPIキーが不要なこと、データが手元のマシンから出ないこと、利用者が増えても推論コストが増えないこと、オフラインで動くことを利点として挙げています。buttercup.sh によく似た構成とされ、「course」「lesson 2」という表現があることから、段階的に学ぶ講座の第1回にあたる教材と読み取れます。ただし Install と Usage は空欄で、説明文にも TODO が残っており、まだ作り始めの段階です。

要点

- WebLLMでブラウザ内完結のAIエージェントを作るための土台
- APIキー不要、データは手元に残り、推論コストはゼロという考え方
- 講座の第1回（week_one）の教材として作られているとみられる
- 実体は buttercup/week_one/index.html の1ファイルのみ
- Install と Usage は未記入で、整備途中の段階

何をするものか

README には「Gateway Repository for building AI Agents using WebLLM. Closely resembles buttercup.sh」と書かれており、WebLLM を使ったAIエージェント開発の入り口になるリポジトリです。本文には「Nothing else in this course does that」「before lesson 2 gives you a model good enough...」といった表現があるため、複数回からなる学習コースの最初の回として位置づけられていると考えられます。ファイル構成も buttercup/week_one/index.html となっていて、週ごとの教材を想定した形です。ただし、buttercup.sh との具体的な関係やコース全体の内容は README からは不明です。

WebLLMを使う理由（README の主張）

Background では、ブラウザ内で LLM を動かす利点を4つ挙げています。1つ目は手間のなさで、サインアップ、APIキーの管理、課金ダッシュボードがなく、何度失敗しても費用がかかりません。2つ目はプライバシーで、モデルの重みはユーザーの GPU に読み込まれ、プロンプトが外部に送られることはありません。そのため、病院や銀行にも見せられるデモになると説明しています。3つ目はコストで、推論は各ユーザーの GPU で行われるため、利用者が何人に増えても費用は変わりません。4つ目はオフライン動作で、一度モデルをダウンロードすれば、ネット接続がなくても応答し続けます。

構成とファイル

ファイル一覧のうち、中身のある成果物は buttercup/week_one/index.html の1つだけです。ほかは .gitattributes、JetBrains 系 IDE の設定である .idea ディレクトリ、README.md となっています。依存関係を示すマニフェスト（package.json など）は存在せず、ビルド工程を持たない単一の HTML ページとして作られている可能性があります。index.html の中で WebLLM をどう読み込み、エージェントをどう組んでいるかは、提示された情報からは確認できません。使用するモデルについては「a 2B model in a tab」という記述があり、20億パラメータ規模の小型モデルを想定しているようですが、具体的なモデル名は README からは不明です。

使い方・導入方法

README の Install と Usage には、空のコードブロックが置かれているだけで、導入手順や実行コマンドは書かれていません。単一の HTML ファイルという構成から、ブラウザで開いたり、ローカルサーバーで配信したりして使う形が想像されますが、正式な手順は README からは不明です。必要なブラウザや GPU の要件、モデルのダウンロードサイズなども記載がありません。実際に試す場合は、index.html の中身を直接読んで確認する必要があります。

注目ポイント・注意点

README は、ブラウザ内の 2B モデルについて「小さく、遅く、自信満々に間違える」と正直に書いています。最先端のモデルには遠く及ばないことを認めたうえで、だからこそエージェントのあらゆる部分が壊れる様子を無料で観察できる、と教育上の意義を説明しているのが特徴です。一方で、説明文に「TODO: Fill out this long description.」が残っています。スター数とフォーク数はともに0で、作り始めの段階です。ライセンスについては、README に「MIT © 2026 Nick Montana」とあるものの、メタデータ上は不明で、ファイル一覧に LICENSE ファイルもありません。

技術構成: HTML、WebLLM
向いている人: APIキーや課金なしでLLMエージェントの仕組みを試したい学習者や学生、ブラウザ内でLLMを動かすWebLLMに興味のあるWeb開発者、プライバシーやオフライン動作を重視したデモを作りたい人に向いています。ただし教材として整備途中のため、現時点では完成した講座ではなく、講座の雛形として見るのが適切です。

テーマ 5. Hazz-i/WebLlmBridge

https://github.com/Hazz-i/WebLlmBridge

★ 0 ／ フォーク 0 ／ 言語 JavaScript ／ 作成 2026-11-01 ／ GPL-3.0

該当テーマ: WebLLM

ブラウザでログイン中の Claude.ai や ChatGPT などの Web チャットを、OpenAI 互換のローカル API として使えるようにするブリッジです。

概要

WebLlmBridge は、Chrome/Edge 拡張機能と Python 製のローカルブリッジ（bridge.py）を組み合わせたツールです。ブラウザで個人アカウントにログインしている Claude.ai や ChatGPT の Web チャットを、DOM 操作で自動入力・送信します。その結果を /v1/chat/completions という OpenAI 互換のエンドポイントとして返します。主な用途は Hermes Agent やローカルのワークフローからの利用で、API トークンを別途契約しなくても Web 版のチャット AI を呼び出せる点が特徴です。

要点

- Web チャットを OpenAI 互換 API（ポート 17614）として公開する
- 拡張機能と bridge.py が WebSocket（17613）で通信する
- Hermes Agent の custom provider として設定できる
- Cloudflare Tunnel 用スクリプトでリモートからも利用できる
- ChatGPT で生成された画像を MEDIA タグで Telegram に渡せる

何をするものか

Claude.ai、ChatGPT、Gemini などのブラウザ版 AI チャットを、外部のプログラムから API のように呼び出すための「Web-to-API」ブリッジです。README は Hermes Agent（NousResearch）との連携を主な用途にしており、Telegram、Discord、CLI 上の Hermes から送ったメッセージがブラウザのチャット画面に入力され、返ってきた応答テキストが Hermes に戻ります。README では、API の利用料を払わずに、ブラウザでログイン済みの個人アカウントのセッションを使えることが利点として挙げられています。README 本文はインドネシア語で書かれています。

仕組み・アーキテクチャ

処理は 4 層で流れます。Hermes Agent が HTTP POST /v1/chat/completions を送ると、ローカルブリッジ bridge.py（ポート 17614）が受け取ります。bridge.py は ws://127.0.0.1:17613 の WebSocket を通して拡張機能に指示を渡し、拡張機能は Web チャットのページを DOM 操作して文章の入力と送信ボタンのクリックを自動で行います。/v1/models が返すモデルは 3 つです。web-claude は claude.ai のタブ、web-chatgpt は chatgpt.com のタブ、web-active-tab は開いている任意の AI タブに振り分けられます。

主な機能

中心となる機能は、OpenAI 互換のローカル API、Hermes Agent との連携、Cloudflare Tunnel 用スクリプトの 3 つです。トンネル用には run_tunnel.sh と run_tunnel.bat が用意されています。これにより、別のマシンにある Hermes サーバーから、手元の PC のブリッジへ trycloudflare.com 経由でアクセスできます。画像生成にも対応しています。ChatGPT Web 上で DALL-E が描画した img 要素を拡張機能が検出すると、bridge.py が元の解像度の画像をローカルキャッシュに保存し、応答に MEDIA:/path/to/image.png というタグを埋め込みます。Hermes のゲートウェイはこのタグを読み取り、Telegram にネイティブの写真として表示します。

技術構成

拡張機能は Manifest V3 の JavaScript で書かれています（snapgent-extension）。providers ディレクトリには chatgpt.js、claude.js、gemini.js、google-flow.js などがあり、サービスごとに処理が分かれています。ブリッジ側は Python 製で、依存ライブラリとして websockets を使います。ほかに snapgent-app ディレクトリがあり、package.json、main.js、preload.js、toolbar.html などが入っています。こちらには deepseek、grok、kimi、mistral、qwen、glm、meta、arena などのプロバイダファイルもあります。デスクトップアプリ版のように見えますが、その位置づけは README からは不明です。vscode_mcp.py や assets/mcpsupport.jpeg もあり、MCP 関連の機能がありそうですが、詳細は README に記載がありません。

使い方・導入方法

手順は次のとおりです。 1. Chrome または Edge で chrome://extensions（Edge は edge://extensions）を開き、Developer mode を有効にします。 2. Load unpacked で snapgent-extension フォルダを読み込みます。 3. Claude.ai または ChatGPT を開き、自分のアカウントでログインします。 4. pip install websockets で依存ライブラリを入れ、python3 bridge.py でブリッジを起動します。ログに extension connected と出れば接続完了です。 5. python3 test_hermes_endpoint.py で動作を確認します。 6. Hermes 側は ~/.hermes/config.yaml で provider を custom、base_url を http://127.0.0.1:17614/v1 に設定します。hermes config set コマンドでも設定できます。

注目ポイント・注意点

API 契約なしで Web 版の AI を自動化できる一方、DOM 操作に頼る仕組みのため、各サービスの画面が変わると動かなくなる可能性があります。個人アカウントのセッションを自動で使う形になるので、各サービスの利用規約に合うかどうかは利用者自身が確認する必要があります。リポジトリには bridge.exe や vscode_mcp.exe といった実行ファイルと、web_tokens.json というファイルも含まれています。web_tokens.json の中身は README からは不明なので、扱いには注意してください。README のアーキテクチャ図には「WebSocket が 0.0.0.0 で待ち受ける」とあり、API は外部からも届く設定です。トンネルで公開するときは認証まわりに気をつける必要がありますが、認証機能があるかどうかは README からは不明です。スター数は 0 で、まだ初期段階のプロジェクトです。ライセンスは GPL-3.0 です。

技術構成: JavaScript、Chrome Extension Manifest V3、Python、websockets、Cloudflare Tunnel、Node.js（snapgent-app、詳細不明）
向いている人: Hermes Agent を使っていて、Claude や ChatGPT の Web 版をバックエンドとして試したい人に向い

ています。ローカルのワークフローから OpenAI 互換 API でブラウザ上の AI を呼び出したい開発者や、ブラウザ自動化と LLM の連携方法を学びたいエンジニア・学生の参考にもなります。

総合 6. deepseek-ai/dsh-superagent-prototype

https://github.com/deepseek-ai/dsh-superagent-prototype

★ 8,297 ／ フォーク 839 ／ 言語 TypeScript ／ 作成 2026-10-31 ／ MIT

DeepSeek Harness 向けバンドルとして動く、単一 GitHub リポジトリ専属の能動型コーディングエージェントの研究プロトタイプ

概要

dsh-superagent は、GitHub と任意で Feishu を監視し、許可された人からの依頼をワークアイテム（Ticket）に変換して、専用のエージェントセッションとディレクトリで作業し、プルリクエストなどで元の会話に返答するエージェントです。チーム全体の代わりに動けるエージェントはチーム全体の注意を浪費しうるという問題意識から、「人間の注意をできるだけ奪わず、できるだけ価値を返す」ことを設計原則に据えています。依頼されたときかチームが書いたルールが許すときだけ動き、誰かが行動すべき時や結果が役立つ時だけ発言するというポリシー駆動の振る舞いが特徴です。DeepSeek の公式製品ではなく、論文 Exploring Super-agent Design に関連する参考用の研究コードとして公開されています。

要点

- DeepSeek Harness (DSH) 上で動くバンドル形式のエージェント
- GitHub と Feishu の依頼を Policy で評価し Ticket 化して実行
- 人間の注意を最小化する思想を docs で明文化
- セットアップ自体をコーディングエージェントに任せる SETUP.md
- 公式製品ではない研究用プロトタイプで本番利用は非推奨

何をするものか

dsh-superagent は、ひとつの GitHub リポジトリに張り付いて働く能動的なコーディングエージェントです。GitHub と、オプションで Feishu（中国発のチャットツール）を監視し、許可されたメンバーから作業依頼があるとワークアイテムを開き、それぞれ独立したエージェントセッションと作業ディレクトリで処理します。結果はプルリクエストなどの形で、依頼が来た元の会話に返されます。人の対応が必要な事項は Web パネルで一覧できます。package.json の説明によれば、依頼は Policy の下で評価され、認可されたものだけが Ticket となり、各 Ticket は専用の DSH Session で実行されます。

設計思想

README が強調するのは「人間の注意をできるだけ少なく取り、できるだけ多くの価値を返す」という働き方です。チームのために動けるエージェントは、裏を返せばチーム全員の注意を浪費しうる存在でもあります。そのためこのエージェントは、依頼されたとき、またはチームが書いたルールが許可したときにのみ行動し、誰かが行動しなければならないときや結果が役立つときだけ発言する、という制約を持ちます。この原則とオペレーターに求められることは docs/000-philosophy.md にまとめられており、ドキュメントは番号順に読むことで前提を積み上げられる構成になっています。

仕組み・アーキテクチャ

ソースは役割ごとに分かれています。src/core には engine、intake、tickets、assignments、memory、replies、stage など Ticket のライフサイクルを扱うドメイン層があり、src/sources には GitHub と Feishu の trigger、channel、read、poll、registry といった入力元の実装が並びます。src/admission.ts と src/policy.ts が依頼の受付判定と認可を担うと考えられます。src/app には outbox、delivery、sessions、runtime、settings など実行基盤、src/executor には DSH 実行器、src/client には React 系 TSX で書かれた Web パネルがあります。prompts ディレクトリには orchestrator、worker、dispatch、handoff など役割別プロンプトがあり、オーケストレーターとワーカーの分担構成がうかがえます。詳細は docs/007-architecture.md に記載されています。

技術構成

主要言語は TypeScript で、Node.js 22.19 以上または 24 以上、パッケージマネージャーは pnpm 11.7.0 が指定されています。ビルドは tsdown、テストは vitest、型チェックは tsc です。直接依存は zod と proper-lockfile のみと少なく、大半は peerDependencies として @deepseek-ai/cordis、@deepseek-ai/dsh-agent、dsh-session、dsh-llm、dsh-tools、dsh-skill、dsh-workspace、dsh-host-webserver など DeepSeek Harness のパッケージ群（0.2.1-alpha.2 系）に依存します。設定は cordis.patch.yml でバンドルとしてパッチされ、locale には英語と中国語の翻訳ファイルが用意されています。tests 配下には多数の spec が揃っています。

使い方・導入方法

セットアップは手順が多いため、エージェントと一緒に行う方式が採られています。手元のコーディングエージェントに「Read SETUP.md in this repository and set up dsh-superagent with me.」と依頼すると、エージェントが短い質問を一つずつ投げ、全体計画を一度提示した後、インストールと設定を行います。手動で導入・設定・アップグレード・トラブルシュートしたい場合は docs/006-deployment.md を参照します。開発時は pnpm check で型チェック、テスト、ビルドを一括実行できます。新しい入力元（Source）の追加方法は docs/008-cookbook.md に解説されています。

注目ポイント・注意点

README 冒頭で、これは公式の DeepSeek 製品ではなく参考用の研究プロトタイプであると明記されています。日々の開発から生まれ、多くのエンジニアリングをコーディングエージェントが担ったため、バグやコーディングスタイルの不統一が残っている可能性があり、利用前にレビューと調整が必要です。また公開用に手を加えているため、社内版や論文で述べられた実装とは同一ではなく、論文中の一部設計は将来の方向性で未実装とされています。依存する DSH パッケージはアルファ版であり、DeepSeek Harness 自体の入手方法や動作環境の詳細は README からは不明です。

技術構成: TypeScript、Node.js、pnpm、tsdown、vitest、zod、proper-lockfile、DeepSeek Harness (DSH)、@deepseek-ai/cordis、@deepseek-ai/schemastery、GitHub、Feishu
向いている人: 自律型・能動型のコーディングエージェントの設計を研究したい人、GitHub リポジトリの Issue や PR 対応をエージェントに任せる仕組みを検討しているチーム、DeepSeek Harness のバンドル開発の実例を知りたい開発者に向いています。研究コードのため、本番運用よりも設計や実装の参考として読む用途が適しています。

総合 7. anhao/ai-pose-gallery-zh

https://github.com/anhao/ai-pose-gallery-zh

★ 5,174 ／ フォーク 1,322 ／ 言語 TypeScript ／ 作成 2026-10-27 ／ NOASSERTION

日本の作者ギア氏のAI絵画用ポーズ・服装・人物参考画像集を中国語化・統合し、生成AI連携ワークベンチとComfyUIプラグインを加えたプロジェクト

概要

原作者ギア（@giaaibusi）が日本語で公開していたポーズ・服装・人物の3つのAI絵画参考図鑑を、1つのサイトに統合して全メタデータを中国語化したリポジトリです。合計4,345件・11,568枚の画像をローカルにミラーしており、オフライン閲覧や静的ホスティングが可能です。さらに、React 19製のWeb合成ワークベンチから Gemini、OpenAI GPT Image、Qwen、Doubao、ローカル ComfyUI の5系統の画像生成APIを直接呼び出せます。リポジトリ自体を ComfyUI の custom_nodes としても使える点が特徴です。

要点

- ポーズ・服装・人物の3図鑑を統合し、4,345件のデータを全て中国語化
- ポーズは素体白モデルと深度図（Depth）を切り替え可能でControlNet向き
- Web合成台から5種のAI画像APIとローカルComfyUIに直結
- git cloneするだけで使えるComfyUIカスタムノード4種を同梱
- 画像の著作権は原作者に帰属し、コードのみMITライセンス

何をするものか

原作者ギア氏が公開している姿勢（ポーズ）、服装、人像の3つのAI絵画用参考図鑑は、元々メタデータが日本語で、しかも3つの独立サイトに分かれていました。本リポジトリ（v2.0）はこれを1つに統合し、全データを中国語へ翻訳したものです。画像は全量をローカルにミラーしています。内訳は、ポーズ白モデル2,878枚、深度図2,878枚、服装2,814枚、人像2,998枚で、合計11,568枚です。そのうえで、参考画像を組み合わせてAIでキャラクターを生成する合成ワークベンチと、ComfyUI プラグインを追加しています。オンラインデモは ai-pose-gallery.anhao.app で公開されています。

主な機能

ポーズ図鑑は4体系・26の動作分類・5つの視点で構成され、白モデルと深度図をワンクリックで切り替えられます。服装図鑑は8分類で、86の中国語タグと英語のPromptキーワードを持ちます。人像図鑑は正面顔と雰囲気の2区分に分かれ、4種の髪の長さ、51種の髪型で絞り込めます。合成台（/studio）では、人像・服装・ポーズの3スロットに任意枚数の追加参考画像を加えられます。組み合わせた画像は1800×1200のCanvasで合成プレビューされます。12種の画風プリセットから中国語の構造化プロンプトを自動で組み立てる機能もあります。生成履歴は IndexedDB に保存され、ページを移動しても生成処理は中断されません。

仕組み・アーキテクチャ

フロントエンドは src/ 以下にまとまっています。routes でページを構成し、services/aiGenerator.ts が5種のAIプロトコルのクライアント、canvasComposer.ts が画像合成、store/studioStore.tsx が状態管理と履歴を担当します。データは data/ にある日本語の原データ（pose.json など）と、中国語化したデータ（pose-zh.json など）のJSONで管理されています。Python スクリプト群がこのデータを扱い、download.py で取得、i18n.py と apply_translation.py で翻訳、verify.py で検証します。Qwen や Doubao の CORS 問題は、Cloudflare Workers / Pages Functions に置いた /api/proxy という同一オリジンのプロキシで回避しています。ComfyUI 側は comfyui_plugin/ 配下にあり、nodes.py にノード実装、routes.py に画像配信ルート、web/extension.js にキャンバス上の選択UIがあります。

技術構成

フロントエンドは React 19.3、TypeScript 7.0、Vite 8（Rolldown）で構築されています。ルーティングとデータ取得は TanStack Router と TanStack Query、UIは Tailwind CSS 4.3、shadcn/ui（Radix UI）、Lucide Icons です。コード品質管理には Oxlint と Oxfmt を使っています。ComfyUI プラグインは Python 製で、aiohttp、Pillow、torch に加え、LiteGraph のフロントエンド拡張を利用します。ComfyUI に同梱されている依存を流用するため、追加のインストールは不要とされています。デプロイ先は Cloudflare Workers / Pages で、wrangler.jsonc と worker.ts が付属しています。さらに skills/ には、Claude Code などのエージェント向けに SKILL.md 形式の検索スキルが8つ収録されています。

使い方・導入方法

ローカル開発には Node.js 20+ と pnpm が必要です。pnpm install を実行した後、pnpm dev で http://localhost:5173 に開発サーバーが立ち上がります。ComfyUI プラグインとして使う場合は、ComfyUI/custom_nodes で git clone --depth 1 を実行して再起動します。右クリックメニューの「AI角色三合一参考图集」から4種のノードを追加できます。Webの合成台からローカルの ComfyUI を呼ぶ場合は、python main.py --enable-cors-header "*" で起動したうえで、設定画面から接続します。ワークフローはAPI形式の .json をインポートでき、{{prompt}} や {{pose_image}} といったプレースホルダーに対応しています。Cloudflare へ公開する場合は、bash scripts/build.sh でビルドしてから npx wrangler deploy を実行します。

注目ポイント・注意点

リポジトリ全体は約1.1GB、ファイル数は約11,600あり、clone には相応の容量が必要です。図鑑には18歳以上向けの素体ポーズや下着の参考画像が含まれますが、既定ではSFWモードで非表示になっています。画像と原データの著作権は原作者ギア氏にあります。MITライセンスが適用されるのは src/、comfyui_plugin/、scripts/ のコードのみで、img/、thumbs/、data/ には適用されません。画像の転売や、無断で実在人物の顔を差し替える用途は明確に禁止されています。メタデータ上のライセンス表記は NOASSERTION です。なお、README に記載されたモデル名（gpt-image-2.5 など）が実際に利用できるかは、README からは検証できません。

技術構成: TypeScript、React 19、Vite 8 (Rolldown)、TanStack Router、TanStack Query、Tailwind CSS 4、shadcn/ui (Radix UI)、Lucide Icons、IndexedDB、Cloudflare Workers / Pages、Python、aiohttp、Pillow、torch、LiteGraph、Oxlint / Oxfmt
向いている人: ポーズや服装の参考画像を探しているイラストレーターやAI画像生成のクリエイターに向いています。

ComfyUI でキャラクター合成ワークフローを組みたいユーザーや、ControlNet Depth 用の素材が欲しい人にも有用です。また、React 19・TanStack・Cloudflare Workers を使った静的ギャラリーと、複数の画像生成APIを統合する実装例を学びたいエンジニアにも参考になります。データは中国語ですが、構成や仕組みは日本の開発者や学生にとっても参考になります。

総合 8. heydamir/unslop

https://github.com/heydamir/unslop

★ 4,954 ／ フォーク 247 ／ 言語 JavaScript ／ 作成 2026-10-29 ／ CC-BY-4.0

AIが作ったっぽいUIを、デザイナーのレビューのように段階的に直して個性あるデザインに仕上げる Claude Code / Codex 向けスキル

概要

unslop は、いわゆる vibecoding で生成された「AIっぽい」UI（紫のグラデーション、グロー、絵文字ラベル、大げさなコピー、不揃いな余白など）を検出し、プロダクトに合った方向性を定めてから一段ずつ修正していく Claude Code と Codex 用のスキルです。Next.js、React、Vue、Svelte、Tailwind、素の HTML/CSS など実装技術を問わず、ダッシュボード、アプリ、ランディングページ、さらには URL しかないライブサイトにも適用できます。編集前に監査と方向性の提案を行う点、良い画面には大改造をせず小さな改善だけを提案する点が特徴です。修正の過程を短い動画にまとめる story video 機能も備えています。

要点

- AIっぽいUIの兆候を監査し、場所付きで一覧化する
- 余白・色・アイコン・コピー・書体など順番にパスを重ねて修正
- 編集前に方向性を提案するので早い段階で舵取りできる
- Playwright と ffmpeg で10秒未満の正方形のビフォーアフター動画を生成
- ライセンスは CC BY 4.0 で、クレジット表記すれば商用利用可

何をするものか

unslop は、AI で手早く作られた画面にありがちな「slop（安っぽい痕跡）」を取り除き、独自の個性を持つデザインへ変えるためのエージェント用スキルです。作者は X で Unslopping シリーズを発信している Damir Sokolovsky 氏です。README によると、まず紫のグラデーション、グロー効果、絵文字ラベル、叫ぶようなコピー、不均一な余白といった兆候を見つけ出し、その後プロダクトの内容から方向性を決めて、デザイナーが赤入れレビューをするように画面を一段ずつ直していきます。すでに良い画面であればそう伝え、全面的な作り直しではなく小さな改善提案にとどめるとされています。

主な機能

機能は大きく5つです。Audit では slop の兆候をその場所とともに列挙します。Direction では、キャラクター、世界観、色を担う1〜2か所の見せ場、シグネチャーとなるディテールをプロダクトから導きます。Passes では spacing、color、icons、copy、type、layout、details、visuals、motion の順に修正します。Visuals では光沢のある抽象的な塊ではなくプロダクト自体を見せる方針をとり、ヒーローに情景やオブジェクトが必要な場合は fal、Replicate、OpenAI images、Higgsfield などの画像サービスで生成できます。未接続なら MCP サーバーとしての接続を提案し、有料生成の前には毎回プロンプトを示して確認します。Story video では画面が改善されていく10秒未満の正方形動画を作ります。

仕組み・構成

ファイル構成を見ると、中核は skills/unslop/SKILL.md で、ここにスキルの手順が書かれていると考えられます。references 配下には signs.md（slop の兆候）、tokens.md、effects.md、visuals.md、refs.md といった参照資料と、改善後の画面例やビジュアル参考画像（webp）が置かれています。scripts には snapshot.mjs と story.mjs があり、名前と README からライブサイトのスナップショット取得と story video 生成を担うものと読み取れます。agents/openai.yaml は Codex 向けの設定と思われますが、詳細は README からは不明です。examples には試用向けの vibecoded な dashboard.html と hero.html が含まれます。

導入方法

Claude Code では git clone した後、skills/unslop を ~/.claude/skills/ にコピーします。単一プロジェクトだけで使う場合はプロジェクト内の .claude/skills/ に置きます。Codex では「$skill-installer install https://github.com/heydamir/unslop/tree/main/skills/unslop」で直接インストールするか、~/.agents/skills/ またはプロジェクト内の .agents/skills/ に手動でコピーします。ライブサイトのスナップショットや story video を使う場合は、スキルを実行するプロジェクトで「npm install -D playwright」と「npx playwright install chromium」を実行し、動画にはさらに ffmpeg を PATH に通しておく必要があります。

使い方

Claude Code では /unslop、Codex では $unslop に続けて依頼内容を書きます。例として「/unslop this dashboard」「/unslop the hero of https://example.com」「/unslop my landing page and make the story video so I can post it」が挙げられています。コマンドを使わず自然な言葉で頼んでも、エージェントが自動でスキルを選ぶとされています。実行するとまず画面を監査し、編集前に方向性を提案するため、ユーザーは早い段階で方針を修正できます。まずは examples フォルダの2つの画面で試すのが手軽です。

注目ポイント・注意点

特定のフレームワークに依存せず、既存のスタックのまま直せる点と、デザインの判断プロセス（監査→方向性→順序立てた修正）をスキルとして形式化している点が興味深いところです。一方、画像生成は外部の有料サービスを使う場合があり、確認は求められるもののコストに注意が必要です。README によればダッシュボードやアプリでは画像生成はあまり必要ないとされています。ライセンスは CC BY 4.0 で、作者のクレジットとリポジトリへのリンクが条件です。なお作者はデザイナーによる有償の unslop サービスも案内しています。

技術構成: JavaScript (Node.js .mjs スクリプト)、Claude Code スキル、Codex スキル、Playwright (Chromium)、ffmpeg、MCP (画像生成サービス連携)、Markdown 参照資料
向いている人: Claude Code や Codex で UI を素早く作っているが、仕上がりが「AIっぽい」と感じている個人開発者やスタートアップのエンジニア。デザイナー不在のチームでダッシュボードやランディングページの見た目を底上げしたい人や、ビフォーアフター動画を SNS に投稿したい人にも向いています。

総合 9. wangzan101/Mika

https://github.com/wangzan101/Mika

★ 4,007 ／ フォーク 741 ／ 言語 Kotlin ／ 作成 2026-10-29 ／ Apache-2.0

話しかけるだけで、Androidのアプリやウェブを裏画面で操作して用事を片付けてくれるオンデバイスAIエージェントです。

概要

Mikaは、自然言語で頼んだ用事をスマホのアプリやウェブを実際に操作して完了させる、Android向けのAIエージェントアプリです。操作は非表示のバックグラウンド画面で行われるため、作業中もユーザーは普段どおりスマホを使えます。アプリごとのAPI連携は使わず、人と同じように画面を見てボタンを探す方式なので、基本的にどのアプリでも動作します。Mika独自のサーバーはなく、自分のモデルAPIキーか ChatGPT・xAI・OpenRouter アカウントでのサインインで動かします。

要点

- 裏画面でアプリを操作するので、作業中もスマホを使える
- アプリ別の連携は不要で、画面を見て操作するため汎用的
- Mika独自のサーバーはなく、モデルは自分で選んで接続する
- Linuxサンドボックスでの Python・Node 実行やスケジュール実行にも対応
- Apache-2.0 のオープンソースで、APK を配布している

何をするものか

Mika は「グループで今日何が話されたか要約して」「30ドル以下で星4.5以上の充電器をカートに入れて」のような一文の依頼を受け、スマホのアプリやウェブを順に操作して用事を終わらせる AI アシスタントです。WeChat の要約、ショッピングサイトでの商品選び、フライト検索、求人サイトでの挨拶送信などのデモが紹介されています。デモには中国で普及しているアプリも多く使われていますが、README によると特定のアプリに依存しているわけではなく、端末上のどのアプリやウェブサイトでも動作するとされています。

主な機能

アプリ操作では、長いリストやチャット履歴を読む、タップや文字入力をする、テキストを共有するといった動作に対応します。ログイン、認証コード、支払いはユーザーに引き継ぎます。ほかに、ログイン状態を保持するデスクトップ相当の内蔵ブラウザ、フローティングカプセルをダブルタップして今の画面を送る機能、時刻や条件を指定した定期実行、実行中の追加指示や中断・再開があります。音声入力、会話ごとの Linux サンドボックスでの Python・Node・shell 実行、PC から CLI + Skill や MCP 経由でタスクを渡す機能も備えています。既定は支払い前にだけ確認する Ask モードで、Autonomous モードへの切り替えもできます。

仕組み・アーキテクチャ

ファイル構成からは、機能(capability)ごとに cap/phone、cap/web、cap/sandbox というパッケージに分かれていることが読み取れます。phone 側には、仮想ディスプレイと見られる AgentDisplay、

MikaAccessibilityService、OCR を担う PhoneOcr、ADB 接続用の AndroidAdb と NsdAdbServiceFinder、AIDL の IMikaShell があります。ワイヤレスデバッグでペアリングしたヘルパーとアクセシビリティを組み合わせて、裏画面を操作する構成と考えられます。web 側には cdp ディレクトリ(DevtoolsTransport、WebDevtools など)があり、Chrome DevTools Protocol 系の仕組みでページを操作しているようです。sandbox には rootfs.tar.xz、ProotDevices、C で書かれた mikarun があり、proot 系の Linux 環境とみられます。使いながら賢くなる点については、好みを記憶し、完了したタスクをスキル化して次回を速く安定させると説明されています。

技術構成

主要言語は Kotlin で、トピックには jetpack-compose が含まれます。テストの一部には Java、サンドボックス用ランチャには C(app/src/main/cpp/mikarun/mikarun.c)が使われています。ビルドは Gradle Kotlin DSL(app/build.gradle.kts)で、README の一部は ./gradlew updateReadme で自動生成されています。モデルは DeepSeek、通義千問、Kimi、Anthropic、Gemini、OpenAI、xAI、Mistral、Groq、Ollama、LM Studio など多数のプリセットがあります。カスタム設定では OpenAI 互換、Anthropic、Gemini、OpenAI Responses の各エンドポイントを指定できます。androidTest 配下にはアクセシビリティ、Web、サンドボックス、UI などの端末テストが充実しています。依存ライブラリの詳細はマニフェストがないため README からは不明です。

使い方・導入方法

Releases から APK をダウンロードしてインストールします。次に、設定のモデル画面でプロバイダを選び、API キーを入力するかアカウントでサインインします。アプリを操作させたい場合は、設定の「Set up phone control」ウィザードに従い、ワイヤレスデバッグのペアリングを一度行います。チャット、ウェブ、ファイル用途だけならこの手順は不要です。PC が手元にあれば、PC 上の AI エージェントに install-with-agent.en.md の手順を渡して導入を任せることもできます。動作要件は、チャット・ウェブ・ファイルが Android 11 以上、アプリ操作が Android 12 以上(14 以上推奨)、サンドボックスが arm64 端末です。UI は日本語にも対応しています。

注目ポイント・注意点

ユーザーの画面を奪わずに裏画面で作業する点と、アプリ別の連携なしで汎用的に操作できる点が大きな特徴です。API キーや記憶は端末内で暗号化保存され、会話内容や画面情報は設定したモデルプロバイダにだけ送られると説明されています。一方で、第三者のアプリやサイトを自動操作すると利用規約に抵触する可能性があると README 自身が注意を促しており、重要な操作では Ask モードを使うよう推奨しています。決済画面やパスワード入力画面などスクリーンショットを禁止している画面は読み取れず、ユーザーへの引き継ぎになります。ワイヤレスデバッグは Wi-Fi の変更や再起動で切れることがあります。内蔵のアプリ知識は中国語で書かれていますが、英語などでの対話も可能です。

技術構成: Kotlin、Jetpack Compose、Java、C、Gradle (Kotlin DSL)、Android Accessibility Service、ADB(ワイヤレスデバッグ)、AIDL、Chrome DevTools Protocol、proot 系 Linux サンドボックス、MCP、OpenAI互換 / Anthropic / Gemini API
向いている人: スマホでの定型作業や情報収集を AI に任せたい Android ユーザーに向いています。また、モバイル向け AI エージェント、アクセシビリティを使った自動操作、端末上のサンドボックス実行の実装を学びたい Kotlin・

Android エンジニアや学生、自分の LLM や API キーでプライバシーを保ちながらエージェントを試したい人にも参考になります。

総合 10. valentderah/totk-pc-recomp

https://github.com/valentderah/totk-pc-recomp

★ 3,245 ／ フォーク 260 ／ 言語 C ／ 作成 2026-10-29 ／ MIT

Switch版『ゼルダの伝説 ティアーズ オブ ザ キングダム』を、エミュレーションではなく静的リコンパイルでWindows/Linux向けネイティブx86-64アプリにするプロジェクト

概要

TotK Recompは、Nintendo Switch向けタイトル『The Legend of Zelda: Tears of the Kingdom』のゲームコードを静的リコンパイルし、Vulkanレンダラーで動くネイティブアプリとしてWindowsとLinuxで動かすファンプロジェクトです。ゲームファイルは含まれず、利用者が自分のSwitchから正規に吸い出したデータをもとに、自分のマシン上でゲームをビルドする方式です。エミュレータを介さないネイティブ実行に加え、高フレームレート、ウルトラワイド、HDR10、FSR 1、エミュレータ向けModやチートへの対応など、PC向けの拡張機能を備えています。また、コードとドキュメントの大半はメンテナの指示のもとでAIコーディングエージェント(Claude Code)が書いているとREADMEに明記されています。

要点

- ARM64のゲームコードをx86-64ネイティブへ静的リコンパイル
- ゲームファイルは同梱されず、自前のダンプからローカルでビルド
- Vulkan 1.3描画でHDR10、FSR 1、ウルトラワイド、最大144fpsに対応
- エミュレータ向けのRomFS/ExeFS ModとAtmosphèreチートを利用可能
- 開発初期段階で、コードの多くはClaude Codeが記述

何をするものか

Nintendo Switch向けの『ゼルダの伝説 ティアーズ オブ ザ キングダム』を、Windows/Linux上でネイティブに動くx86-64アプリケーションへ変換する静的リコンパイルプロジェクトです。CPU命令を実行時に逐次解釈するエミュレータとは違い、ゲームの実行コードを事前にホスト向けのコードへ変換してからビルドします。対応するのは基本ゲームと、アップデート1.1.0、1.2.1、1.4.3です。1.2.1と1.4.3のセーブデータは相互に引き継げます。ゲームファイルは一切含まれていません。生成物はゲーム本体を含むため、自分専用として使い、配布しないよう強く注意書きされています。ステータスは開発初期で、プレイは可能ですが、表示の不具合やクラッシュはまだ起こり得るとされています。

主な機能

フレームレートは標準で60fpsで、実験的に144fpsまたはモニタのリフレッシュレートまで引き上げられます。グラフィック面では、1080pや2Kなどのカスタム解像度、スーパーサンプリング、AMD FSR 1アップスケーリング、最大16倍の異方性フィルタリングに対応します。HDR10出力では、ピーク輝度・ゲーム画面・HUDの明るさを個別に設定できます。画面比率は16:10(Steam Deckなど)、21:9、32:9に対応し、カメラとUIもその比率に合わせて調整されます。入力はキーボード、マウス、ゲームパッドで、リマップ、振動、ジャイロに対応し、Xboxのボタン表記も表示できます。ほかに、F1の設定メニュー、F10のパフォーマンスオーバー

レイ、インストール時のシェーダー事前準備があり、エミュレータ向けModとAtmosphèreチートも使えます。

仕組み・アーキテクチャ

ファイル構成は大きく3つに分かれます。1つ目のrecompディレクトリにはPython製の変換器があります。nso.pyでSwitchの実行形式NSOを読み込み、lifter.py、jumptables.py、simd_c.py、gen.pyなどでCコードを生成していると見られます。バージョンごとのパッチ定義(patches-v110.txtなど)もここに置かれています。2つ目のruntimeディレクトリはC製の実行環境です。kernel.c、svc.c、ipcなどのSwitchシステム層、nvn_api_*.cでのNVNグラフィックスAPIのVulkan変換、ASTCテクスチャのデコード、オーディオ、入力、Mod、チートを担当しています。3つ目のinstallerディレクトリはPython製のセットアップツールで、ゲームファイルの抽出とビルドを行います。詳細はdocs/architecture.mdにあるとされていますが、本文の内容はREADMEからは不明です。

技術構成

主要言語はCです。リコンパイラ、インストーラ、ビルド・検証ツールにはPythonが使われ、設定メニューなどのUIはC++とDear ImGuiで書かれています。描画はVulkan 1.3で、Vulkan Memory Allocatorを利用します。FSR、HDRトーンマッピング、ASTCデコード、BC3エンコードなどはGLSLシェーダーとして同梱されています。ウィンドウと入力にはSDL 3を使い、Windows向けのネイティブ実装も用意されています。ほかに、音声のlibopus、動画のlibvpx、圧縮のZstandard、ファイル抽出のhactoolを利用しています。クレジットにはRyujinx、deko3d、LAS Recomp、SavageWindRecompなど先行プロジェクトの名前が並び、Switchbrewコミュニティの知見も活用されています。

使い方・導入方法

Windowsの動作要件は、Windows 10以降、SSE4.2対応のx86-64 CPU(AVX2/FMA対応機ではより高速なビルドになる)、Vulkan 1.3とVK_EXT_external_memory_hostに対応したGPU、そしてセットアップ時に約36GiB(完了後は約20GiB)の空き容量です。手順は、ReleasesからTotK-Setup-<version>-windows.zipを入手して展開し、TotK-Setup.exeを起動します。ゲームファイル、言語、インストール先を選んでStartを押すとゲームがビルドされ、完了後はtotk.exeで遊べます。LinuxではLinux 6.7以降とglibc 2.34以降が必要です。AppImageにchmod +xで実行権限を付けて起動します。Steam DeckではDesktop Modeで実行します。アップデートは、新しいセットアップを同じフォルダに対して実行するだけで、セーブと設定は保持されます。設定はtotk.iniでも変更できます。ソースからのビルド方法はdocs/building.mdにまとめられています。

注目ポイント・注意点

大規模な商用タイトルを静的リコンパイルでPCネイティブ化し、NVNからVulkanへの変換やASTCテクスチャのGPUデコードまで実装している点は、技術的に大きな見どころです。Modの扱いも特徴的で、ExeFSのコードパッチ(.ips、.pchtxt)を入れるとゲームが自身を再ビルドします。注意点もいくつかあります。まず、ゲームファイルの入手方法は一切案内されず、自分の本体から正規に吸い出したデータが必須です。プロジェクト自体のコードはMITライセンスですが、任天堂のゲームに対する権利は何も付与されません。また、任天堂とは無関係の非商用ファンプロジェクトです。さらに、コードの多くがAIエージェントによって書かれており、そのルールはAGENTS.mdに定められています。メンテナがすべてのビルドを実際にプレイし、マージの可否を判断する運用だと説明されています。

技術構成: C、C++、Python、Vulkan 1.3、GLSL、SDL 3、Dear ImGui、Vulkan Memory Allocator、AMD FSR 1、libopus、libvpx、Zstandard、hactool
向いている人: 正規に所持するTotKを、PCの高解像度・高フレームレート・ウルトラワイド・HDR環境やSteam Deckで遊びたいプレイヤーに向いています。また、静的リコンパイル、Switchのシステム層の再実装、NVNからVulkanへのグラフィックス変換といった低レイヤー技術を学びたいエンジニアや学生にとって、貴重な実例です。AIエージェント主体で大規模なCプロジェクトを開発する運用事例として関心がある人にも参考になります。